AI ASSISTED CODING

ASSIGNMENT - 12.5

NAME: R.Teeneswari

HL-NO: 2303A51932 **BATCH:** 30

Lab 12.5:

Task-1:

```
def merge_sort(arr):
    """
    Sorts a list using the Merge Sort algorithm.

    Time Complexity:
    Best Case: O(n log n)
    Average Case: O(n log n)
    Worst Case: O(n log n)
    Space Complexity:
    O(n) auxiliary space
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
```

```
    return result

# Test

data = [38, 27, 43, 3, 9, 82, 10]

print("Sorted:", merge_sort(data))
```

Output:

```
S C:\Users\SAI\OneDrive\Attachments\AI ASSISTED CODING>
orted: [3, 9, 10, 27, 38, 43, 82]
```

Task-2:

```
def binary_search(arr, target):

    """

    Performs binary search on a sorted list.


    Returns index of target if found, otherwise -1.

    Time Complexity:

    Best Case: O(1)

    Average Case: O(log n)

    Worst Case: O(log n)

    Space Complexity:

    O(1)

    """

    low = 0

    high = len(arr) - 1

    while low <= high:

        mid = (low + high) // 2

        if arr[mid] == target:

            return mid

        elif arr[mid] < target:

            low = mid + 1

        else:

            high = mid - 1

    return -1

# Test
```

```
numbers = [2, 5, 7, 9, 12, 15]

print("Index:", binary_search(numbers, 9))
```

Output:

```
PS C:\Users\SAI\OneDrive\Attachments\AI
Index: 3
```

Task-3:

```
appointments = [

    {"id":101,"patient":"Ram","doctor":"Dr.A","time":10,"fee":500},

    {"id":102,"patient":"Sita","doctor":"Dr.B","time":12,"fee":300},

    {"id":103,"patient":"Arjun","doctor":"Dr.C","time":9,"fee":700}

]


# Sort by time

sorted_by_time = sorted(appointments, key=lambda x: x["time"])

print("Sorted by Time:", sorted_by_time)

# Search by ID

def search_appointment(data, aid):

    for a in data:

        if a["id"] == aid:

            return a

    return "Not Found"

print(search_appointment(appointments,102))
```

Output:

```
Sorted by Time: [{'id': 103, 'patient': 'Arjun', 'doctor': 'Dr.C', 'time': 9, 'fee': 700}, {'id': 101, 'patient': 'Ram', 'doctor': 'Dr.A', 'time': 10, 'fee': 500}, {'id'
: 102, 'patient': 'Sita', 'doctor': 'Dr.B', 'time': 12, 'fee': 300}]
{'id': 102, 'patient': 'Sita', 'doctor': 'Dr.B', 'time': 12, 'fee': 300}
```

Task-4:

```
tickets = [

    {"ticket_id":1,"name":"Raj","train":1201,"seat":45,"date":20240610},

    {"ticket_id":2,"name":"Anu","train":1103,"seat":12,"date":20240608},

    {"ticket_id":3,"name":"Kiran","train":1502,"seat":30,"date":20240612}

]
```

```python
# Sort by travel date
sorted_tickets = sorted(tickets, key=lambda x: x["date"])
print("Sorted by Date:", sorted_tickets)
# Search ticket
def search_ticket(data, tid):
    for t in data:
        if t["ticket_id"] == tid:
            return t
    return "Ticket not found"
print(search_ticket(tickets,2))
```

Output:

```
PS C:\Users\SAI\OneDrive\Attachments\AI ASSISTED CODING> & C:/Python314/python.exe "c:/Users/SAI/OneDrive/Attachments/AI ASSISTED CODING/ASS 12.py"
Sorted by Date: [{'ticket_id': 2, 'name': 'Anu', 'train': 1103, 'seat': 12, 'date': 20240608}, {'ticket_id': 1, 'name': 'Raj', 'train': 1201, 'seat': 45, 'date': 2024061
0}, {'ticket_id': 3, 'name': 'Kiran', 'train': 1502, 'seat': 30, 'date': 20240612}]
{'ticket_id': 2, 'name': 'Anu', 'train': 1103, 'seat': 12, 'date': 20240608}
```

Task-5:

```python
rooms = [
    {"student_id":1,"room":205,"floor":2,"date":20240110},
    {"student_id":2,"room":101,"floor":1,"date":20240105},
    {"student_id":3,"room":305,"floor":3,"date":20240115}
]

# Sort by room number
sorted_rooms = sorted(rooms, key=lambda x: x["room"])
print("Sorted by Room:", sorted_rooms)
# Search student allocation
def search_student(data, sid):
    for r in data:
        if r["student_id"] == sid:
            return r
    return "Record not found"
print(search_student(rooms,3))
```

Output:

```
PS C:\Users\SAI\OneDrive\Attachments\AI ASSISTED CODING> & C:/Python314/python.exe "c:/Users/SAI/OneDrive/Attachments/AI ASSISTED CODING/ASS 12.py"
Sorted by Room: [{'student_id': 2, 'room': 101, 'floor': 1, 'date': 20240105}, {'student_id': 1, 'room': 205, 'floor': 2, 'date': 20240110}, {'student_id': 3, 'room': 30
5, 'floor': 3, 'date': 20240115}]
{'student_id': 3, 'room': 305, 'floor': 3, 'date': 20240115}
```

Task-6:

```
movies = [
    {"id":1,"title":"Inception","genre":"Sci-Fi","rating":8.8,"year":2010},
    {"id":2,"title":"Avatar","genre":"Sci-Fi","rating":7.9,"year":2009},
    {"id":3,"title":"Joker","genre":"Drama","rating":8.5,"year":2019}
]

# Sort by rating
sorted_movies = sorted(movies, key=lambda x: x["rating"], reverse=True)
print("Sorted by Rating:", sorted_movies)
# Search movie
def search_movie(data, mid):
    for m in data:
        if m["id"] == mid:
            return m
    return "Movie not found"
print(search_movie(movies,1))
```

Output:

```
PS C:\Users\SAI\OneDrive\Attachments\AI ASSISTED CODING> & C:/Python314/python.exe "c:/Users/SAI/OneDrive/Attachments/AI ASSISTED CODING/ASS 12.py"
Sorted by Rating: [{'id': 1, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'year': 2010}, {'id': 3, 'title': 'Joker', 'genre': 'Drama', 'rating': 8.5, 'year':
2019}, {'id': 2, 'title': 'Avatar', 'genre': 'Sci-Fi', 'rating': 7.9, 'year': 2009}]
{'id': 1, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'year': 2010}
```

Task-7:

```
crops = [
    {"id":1,"name":"Wheat","moisture":40,"temp":25,"yield":3.5},
    {"id":2,"name":"Rice","moisture":60,"temp":28,"yield":4.2},
    {"id":3,"name":"Corn","moisture":35,"temp":27,"yield":3.0}
]

# Sort by moisture level
sorted_crops = sorted(crops, key=lambda x: x["moisture"])
print("Sorted by Moisture:", sorted_crops)
```

```python
# Search crop

def search_crop(data, cid):

    for c in data:

        if c["id"] == cid:

            return c

    return "Crop not found"

print(search_crop(crops,2))
```

Output:

```
PS C:\Users\SAI\OneDrive\Attachments\AI ASSISTED CODING> & C:/Python314/python.exe "c:/Users/SAI/OneDrive/Attachments/AI ASSISTED CODING/ASS 12.py"
Sorted by Moisture: [{'id': 3, 'name': 'Corn', 'moisture': 35, 'temp': 27, 'yield': 3.0}, {'id': 1, 'name': 'Wheat', 'moisture': 40, 'temp': 25, 'yield': 3.5}, {'id': 2,
 'name': 'Rice', 'moisture': 60, 'temp': 28, 'yield': 4.2}]
{'id': 2, 'name': 'Rice', 'moisture': 60, 'temp': 28, 'yield': 4.2}
```